

[Dashboard](#) / [My courses](#) / [COMS3005A-AAA3-S2-2024](#) / [Quizzes](#) / [Quiz 3 Classical Search and Sorting](#)**Started on** Saturday, 14 September 2024, 7:36 PM**State** Finished**Completed on** Saturday, 14 September 2024, 7:48 PM**Time taken** 11 mins 55 secs**Grade** 27.00 out of 30.00 (90%)Question **1**

Correct

Mark 1.00 out of 1.00

Consider the linear search algorithm shown below:

Algorithm 1 $\text{linearSearch}(myList, n, key)$

Input: $myList, n, key$ where $myList$ is an array with n entries (indexed $0 \dots (n - 1)$), and key is the item sought.*Output:* $index$ the location of key in $myList$.01 $index \leftarrow 0$ 02 While $key \neq myList[index]$ 03 $index \leftarrow index + 1$ 04 Return $index$

The best case for this algorithm is when the key is in the first position in $myList$.

Select one:

☒ True ✓☐ False

The correct answer is 'True'.

Question **2**

Partially correct

Mark 2.00 out of 3.00

Consider the linear search algorithm shown below:

Algorithm 1 linearSearch(*myList*, *n*, *key*)

Input: *myList*, *n*, *key* where *myList* is an array with *n* entries (indexed $0 \dots (n - 1)$), and *key* is the item sought.

Output: *index* the location of *key* in *myList*.

01 *index* $\leftarrow 0$

02 While *key* $\neq$ *myList*[*index*]

03 *index* $\leftarrow$ *index* + 1

04 Return *index*

Indicate which statements are true for this algorithm

- ☒ a. The algorithm relies on key being in the list. ✓
- ☒ b. The algorithm is $O(n)$ in the worst case. ✓
- ☒ c. If there are two or more numbers with the value of key in the list then the algorithm will not run correctly. ✗
- ☐ d. The algorithm will only work if the list of numbers is not sorted.
- ☐ e. If there are two or more numbers with the value of key in the list then the algorithm will return the position of the first such number.

Your answer is partially correct.

You have correctly selected 2.

The correct answers are:

The algorithm relies on key being in the list.,

If there are two or more numbers with the value of key in the list then the algorithm will return the position of the first such number.,

The algorithm is $O(n)$ in the worst case.

Question **3**

Correct

Mark 1.00 out of 1.00

Suppose the list below is used as input to the bisection search algorithm.

[12, 13, 23, 34, 45, 56, 76, 87, 93, 111, 123, 134, 157, 234, 342, 347, 369]

Searching for which number in the list would result in the best case run time for the search?

Answer: ✓

The correct answer is: 93

Question **4**

Correct

Mark 1.00 out of 1.00

Suppose the list below is used as input to the bisection search algorithm.

[12, 13, 23, 34, 45, 56, 76, 87, 93, 111, 123, 134, 157, 234, 342, 347, 369]

Give one number in the list that would result in the worst case run time for the search?

Answer:



The correct answer is: 87

Question **5**

Correct

Mark 1.00 out of 1.00

Suppose the list below is used as input to the bisection search algorithm.

[12, 13, 23, 34, 45, 56, 76, 87, 93, 111, 123, 134, 157, 234, 342, 347, 369]

For how many numbers in the list would 2 comparisons be required?

Answer:



The middle of the two lists could possibly be searched if the key is not found in the middle of the original list.

This would actually be three comparisons

The original answer was wrong

The correct answer is: 2

Question **6**

Correct

Mark 3.00 out of 3.00

Consider the algorithm shown below:

linearSearch3(myList, n, key)

Input: *myList*, *n*, *key* where *myList* is an array with *n* entries (indexed $0 \dots (n - 1)$), and *key* is the item sought.

Output: *index* the location of *key* in *myList*.

01 $index \leftarrow n - 1$

02 While $key \neq myList[index]$

03 $index \leftarrow index - 1$

04 Return *index*

Indicate which of the statements below apply to the algorithm.

- ☐ a. All of the other answers
- ☒ b. Its best case running time is $O(1)$ ✓
- ☒ c. It assumes that the key is in the list. ✓
- ☒ d. Its worse case running time is $O(n)$ ✓
- ☐ e. It will work correctly if key is not in the list

Your answer is correct.

The correct answers are:

It assumes that the key is in the list.,

Its worse case running time is $O(n)$

,

Its best case running time is $O(1)$

Question **7**

Correct

Mark 1.00 out of 1.00

Consider the algorithm shown below.

algorithm(*myList*, *n*)

Input: *myList*, *n* where *myList* is an array with *n* entries (indexed $0 \dots (n - 1)$)*Output*: *myList*00 *max* $\leftarrow$ maximum number in *myList*01 For *i* from 0 to *max*02 *h*[*i*] $\leftarrow$ 003 For *i* from 0 to *n* - 104 *t* $\leftarrow$ *myList*[*i*]05 *h*[*t*] $\leftarrow$ *h*[*t*] + 106 *i* = 007 For *j* from 0 to *max*08 If *h*[*j*] > 009 For *k* from 1 to *h*[*j*]10 *myList*[*i*] $\leftarrow$ *j*11 *i* $\leftarrow$ *i* + 112 Return *myList*

This is an example of a sorting algorithm that does not work on the basis of comparison of list elements.

Select one:

☒ True ✓☐ False

it doesn't ever directly compare list elements and yet it sorts 😊

The correct answer is 'True'.

Question 8

Correct

Mark 3.00 out of 3.00

Consider the algorithm shown below.

algorithm(*myList*, *n*)

Input: *myList*, *n* where *myList* is an array with *n* entries (indexed $0 \dots (n - 1)$)

Output: *myList*

```
00  max ← maximum number in myList
01  For i from 0 to max
02    h[i] ← 0
03  For i from 0 to n − 1
04    t ← myList[i]
05    h[t] ← h[t] + 1
06  i = 0
07  For j from 0 to max
08    If h[j] > 0
09      For k from 1 to h[j]
10        myList[i] ← j
11        i ← i + 1
12  Return myList
```

Indicate which of the statements below are true for this algorithm.

- ☐ a. The running time of the algorithm is $O(n^2)$
- ☒ b. The algorithm will sort any list of numbers ✖
- ☒ c. The running time of the algorithm is $O(\text{maximum}(n, \text{max}))$ ✔
- ☒ d. The algorithm is not in place ✔
- ☒ e. The algorithm must know the biggest number that could appear in the list ✔

Your answer is correct.

The correct answers are:

The algorithm is not in place,

The running time of the algorithm is $O(\text{maximum}(n, \text{max}))$

,

The algorithm must know the biggest number that could appear in the list

Question 9

Correct

Mark 3.00 out of 3.00

Consider the Insertion Sort algorithm as given below

Algorithm 5 insertionSort(*myList*, *n*)

The insertion sort algorithm

Input: *myList*, *n* where *myList* is an array with *n* entries (indexed $0 \dots n - 1$)

Output: *myList* where the values in *myList* are such that

$myList[0] \leq myList[1] \leq \dots \leq myList[n - 2] \leq myList[n - 1]$

```

01 For i from 1 to n - 1
02   x ← myList[i]
03   j ← i - 1
04   While (j ≥ 0) and myList[j] > x
05     myList[j + 1] ← myList[j]
06     j ← j - 1
07   myList[j + 1] ← x
08 Return myList

```

Suppose this algorithm is used to sort the list of numbers below.

[9, 5, 6, 7, 1, 3, 2, 8]

The first number that will be moved into its correct place in the growing sorted sublist is ✓ .

After the second execution of the outer loop (the for loop on line 1), the sorted sublist will be ✓ .

During the fourth execution of the for loop, ✓ comparisons will be made between *x* and a list element.

Your answer is correct.

The correct answer is:

Consider the Insertion Sort algorithm as given below

Algorithm 5 insertionSort(*myList*, *n*)

The insertion sort algorithm

Input: *myList*, *n* where *myList* is an array with *n* entries (indexed $0 \dots n - 1$)

Output: *myList* where the values in *myList* are such that

$myList[0] \leq myList[1] \leq \dots \leq myList[n - 2] \leq myList[n - 1]$

```

01 For i from 1 to n - 1
02   x ← myList[i]
03   j ← i - 1
04   While (j ≥ 0) and myList[j] > x
05     myList[j + 1] ← myList[j]
06     j ← j - 1
07   myList[j + 1] ← x
08 Return myList

```

Suppose this algorithm is used to sort the list of numbers below.

[9, 5, 6, 7, 1, 3, 2, 8]

The first number that will be moved into its correct place in the growing sorted sublist is [5].

After the second execution of the outer loop (the for loop on line 1), the sorted sublist will be [[5, 6, 9,...]].

During the fourth execution of the for loop, [4] comparisons will be made between *x* and a list element.

Question **10**

Correct

Mark 1.00 out of 1.00

Imagine we have a list that is in reverse sorted order. Select the sorting algorithm below with the best (smallest) asymptotic time complexity.

- ☐ a. Quicksort (selecting the rightmost value as the pivot)
- ☒ b. Mergesort ✓
- ☐ c. Bubblesort
- ☐ d. Insertion sort

Your answer is correct.

The correct answer is:
Mergesort

Question **11**

Correct

Mark 1.00 out of 1.00

Imagine we have a list that is in sorted order. Which of the sorting algorithms below would complete in the shortest amount of time?

- ☒ a. Insertion sort ✓
- ☐ b. Selection sort
- ☐ c. Max sort
- ☐ d. Mergesort

Your answer is correct.

The correct answer is: Insertion sort

Question 12

Correct

Mark 1.00 out of 1.00

Suppose we have an unsorted list as below

[9, 4, 6, 7, 1, 3, 5, 8]

This list is used as input to the `mergesort` algorithm called with the command

`mergesort(0,7)`

How many instantiations of the recursive algorithm will occur when sorting the list?

Answer: 15



The correct answer is: 15

Question 13

Incorrect

Mark 0.00 out of 2.00

Suppose we have an unsorted list as below

[9, 4, 6, 7, 1, 3, 5, 8]

This list is used as input to the `mergesort` algorithm called with the command

`mergesort(0,7)`

What will the `mylist` and `temp` arrays contain once the recursive instantiation `mergesort(0,3)` completes and returns to its calling instantiation.

`mylist = [4, 9, 6, 7, 1, 3, 5, 8] , temp = [9, 4, 6, 7, 1, 3, 5, 8]`



What will the `mylist` and `temp` arrays contain once the recursive instantiation `mergesort(0,1)` completes and returns to its calling instantiation.

`mylist = [4, 6, 7, 9, 1, 3, 5, 8], temp = [4, 9, 7, 6, 1, 3, 5, 8]`



Your answer is incorrect.

The correct answer is:

What will the `mylist` and `temp` arrays contain once the recursive instantiation `mergesort(0,3)` completes and returns to its calling instantiation.

→ `mylist = [4, 6, 7, 9, 1, 3, 5, 8], temp = [4, 9, 7, 6, 1, 3, 5, 8],`

What will the `mylist` and `temp` arrays contain once the recursive instantiation `mergesort(0,1)` completes and returns to its calling instantiation.

→ `mylist = [4, 9, 6, 7, 1, 3, 5, 8] , temp = [9, 4, 6, 7, 1, 3, 5, 8]`

Question **14**

Correct

Mark 1.00 out of 1.00

Suppose we have an unsorted list as below

[9, 4, 6, 7, 1, 3, 5, 8]

This list is used as input to the `quicksort` algorithm called with the first call to the function being `quicksort(0,7)`

How many instantiations of the recursive algorithm will occur when sorting the list?

Answer:



The correct answer is: 9

Question **15**

Correct

Mark 1.00 out of 1.00

Suppose we have an unsorted list as below

[9, 4, 6, 7, 1, 3, 5, 8]

This list is used as input to the `quicksort` algorithm called with the first call to the function being `quicksort(0,7)`

What is the third number that is put into its correct position in the list by the partition operation?

Answer:



The correct answer is: 4

Question **16**

Correct

Mark 1.00 out of 1.00

An in-place algorithm is an algorithm that does not need an extra space and produces an output in the same memory that contains the data by transforming the input "in-place". However, a small constant extra space used for variables is allowed.

Select all of the algorithms below that are "in-place".

- ☒ a. Quicksort ✓
- ☒ b. Bubblesort ✓
- ☒ c. Selection sort ✓
- ☐ d. Mergesort
- ☒ e. Insertion sort ✓

Your answer is correct.

The correct answers are:

Bubblesort,

Selection sort,

Insertion sort,

Quicksort

Question **17**

Correct

Mark 1.00 out of 1.00

Assume that we wish to sort an array, but we know that the array is either already sorted, or very nearly sorted. Which of the algorithms below would be the most preferred choice of sort?

- ☒ a. Insertion sort ✓
- ☐ b. Mergesort
- ☐ c. Selection sort
- ☐ d. Quicksort

Your answer is correct.

The correct answer is:

Insertion sort

Question **18**

Correct

Mark 1.00 out of 1.00

An online sort algorithm is one that can sort a list without requiring access to the entire list up front. In other words, each value in the list can be provided to the algorithm one after the other, and it will still function correctly. Of the algorithms below, select the ones that can be classified as *online*:

- ☐ a. Mergesort
- ☐ b. Bubblesort
- ☒ c. Insertion sort 
- ☐ d. Quicksort

Your answer is correct.

The correct answer is:
Insertion sort

Question 19

Correct

Mark 3.00 out of 3.00

Suppose we have a sorted list of numbers where we know that the range of numbers is small and there are many repeats of each number.

We want to be able to efficiently search this list to determine whether a given number, the key, appears in the list.

One of our friends suggests using a variation of the pigeonhole sort.

We make a first pass over the full list and create the `hole` list (as in pigeonhole sort). Then we directly access the new list to determine whether the hole that should be nonzero, if key is in the original list, is, in fact, non zero.

The algorithm is shown below.

```

00  Algorithm pigeonholeSearch(myList, n)
    Input: myList, n, min, max where myList is a sorted array
           with n entries (indexed 0...n - 1) .
           min is the smallest number in the list and
           max is the largest number in the list
           key is the number being searched for.
    Output: yes if the number is in the original list.  else no
01      For j from min to max
02          hole[j]  $\leftarrow$  0
03      For i from 0 to n - 1
04          t  $\leftarrow$  myList[i]
05          hole[t]  $\leftarrow$  hole[t] + 1
06      If hole[key] > 0
07          Then
08              Return Yes
09      Then
10          Return No

```

This algorithm  have different best and worst cases.

The algorithm runs in  time in the worst case.

The algorithm is  efficient than binary/bisection search on this input.

Your answer is correct.

The correct answer is:

Suppose we have a sorted list of numbers where we know that the range of numbers is small and there are many repeats of each number.

We want to be able to efficiently search this list to determine whether a given number, the key, appears in the list.

One of our friends suggests using a variation of the pigeonhole sort.

We make a first pass over the full list and create the `hole` list (as in pigeonhole sort). Then we directly access the new list to determine whether the hole that should be nonzero, if key is in the original list, is, in fact, non zero.

The algorithm is shown below.

```
00  Algorithm pigeonholeSearch(myList, n)
    Input:  myList, n, min, max where myList is a sorted array
           with n entries (indexed 0...n - 1) .
    min is the smallest number in the list and
    max is the largest number in the list
    key is the number being searched for.
    Output: yes if the number is in the original list.  else no
01      For j from min to max
02          hole[j]  $\leftarrow$  0
03      For i from 0 to n - 1
04          t  $\leftarrow$  myList[i]
05          hole[t]  $\leftarrow$  hole[t] + 1
06      If hole[key] > 0
07          Then
08              Return Yes
09          Then
10              Return No
```

This algorithm [does not] have different best and worst cases.

The algorithm runs in [$O(n)$] time in the worst case.

The algorithm is [less] efficient than binary/bisection search on this input.

[◀ Quiz 2 for 2024 -- AI search](#)

Jump to...

[Quiz 4 Greedy Algorithms, Dynamic Programming and Closest Pair ▶](#)